학습 노트: Dev2 파이프라인 2 (취합 → 코멘트 → GitHub → Slack)

작성일: 2026-06-03 **관련 파일**: `dev2/aggregate_graph.py`, `dev2/nodes/aggregate.py`, `dev2/nodes/comment_refine.py`, `dev2/api/aggregate.py`, `dev2/schema.py`, `dev2/db.py`, `dev2/migrations/004_aggregate_table.sql` **복습 예정**: D+1, D+3, D+7 **작성 방식**: `learning-mentor.md` 에이전트 형식(4-Layer 설명법) 적용

한 줄 요약

여러 검사 로봇이 따로 찾아낸 코드 문제들을 **한곳에 모아 정리**하고, 신입 개발자 눈높이에 맞춰 **리뷰 글을 자동으로 써서** GitHub에 달고, 위험한 문제가 있으면 **선배에게 Slack으로 알리는** 자동 흐름이다.

---

0. 큰 그림 — 이 파이프라인이 하는 일

우리 플랫폼은 PR(코드 변경 요청)이 올라오면 여러 AI 에이전트가 **동시에** 코드를 검사한다.

- **M2(Dev2)**: 정적 분석 + 보안 스캔
- **M3(Dev3)**: 테스트 검증 + 성능 분석

이렇게 따로따로 나온 검사 결과를 **누군가는 모아서 정리**하고, 사람이 읽을 리뷰로 바꿔야 한다. 그 "모으고 → 글쓰고 → 전달하는" 단계가 바로 **파이프라인 2**다. (PRD 문서의 노드 I→M→N→O)

> 비유: 병원에서 피검사실·엑스레이실·심전도실이 각자 결과지를 낸다(파이프라인 1). 그걸 **종합해서 진단서 한 장으로 써주고, 환자 수준에 맞게 설명하고, 위급하면 응급실에 바로 연락하는** 담당의 역할이 파이프라인 2다.

---

1. 워크플로우 (전체 흐름)

```
POST /aggregate              ← M0(오케스트레이터)가 M2+M3 결과를 묶어서 보냄
   │
   ▼
[aggregate]      노드 I  → 모든 이슈 취합 + Critical→Major→Minor 정렬
   │                       + PostgreSQL 저장 + M4(커리큘럼)에 push
   ▼
[comment_refine] 노드 M  → LLM이 junior/mid/senior 수준별 PR 코멘트(마크다운) 생성
   │
   ▼
[github_comment] 노드 N  → GitHub PR에 코멘트 등록 → 코멘트 URL 받아옴
   │
   ▼
   ◇ Critical 이슈 ≥ 1건?  ── 조건부 분기
   │
   ├─ 예 → [slack_notify] 노드 O → 시니어에게 Slack 알림(코멘트 URL 포함) → END
   │
   └─ 아니오 ──────────────────────────────────────────────────────► END
```

핵심 포인트 2가지

1. **조건부 분기**: 모든 PR에 Slack을 보내지 않는다. Critical이 있을 때만 보낸다. 이게 "단순 직선 파이프라인"과 "AI Native Agentic Workflow"를 가르는 지점이다 — 상태(데이터)를 보고 실행 경로가 바뀐다.
2. **github → slack 순서**: GitHub 코멘트를 먼저 달아 그 URL을 받아온 뒤, Slack 알림에 "PR 코멘트 보기" 링크로 끼워 넣는다.

---

2. 4-Layer 설명 — 이번에 쓴 핵심 기술 5가지

2-1. LangGraph StateGraph (로봇들의 작업 흐름 지도)

비유: 공장 컨베이어 벨트 설계도. "1번 기계 → 2번 기계 → (불량이면 검수대로, 아니면 포장대로)" 같은 흐름을 그림으로 그려둔 것.

개념: 노드(작업 단위)와 엣지(이동 경로)로 작업 흐름을 정의하는 그래프. 노드 사이를 **State(공유 상태)**라는 상자가 돌아다니며 각 노드가 상자에 결과를 채워 넣는다.

왜 쓰나: if/else로 비동기 함수를 줄줄이 호출하면, 흐름이 복잡해질수록 "지금 어디서 뭐가 실행되는지" 추적이 불가능해진다. StateGraph는 흐름을 **데이터(그래프 정의)로 분리**해서, 흐름과 로직을 따로 관리하게 해준다. + LangSmith로 자동 추적된다.

실제 코드 (`aggregate_graph.py`):

```python
g = StateGraph(AggregateState)
g.add_node("aggregate",      aggregate_node)
g.add_node("comment_refine", comment_refine_node)
g.add_node("github_comment", github_comment_node)
g.add_node("slack_notify",   slack_notify_node)

g.set_entry_point("aggregate")
```

```
g.add_edge("aggregate", "comment_refine")
g.add_edge("comment_refine", "github_comment")
g.add_conditional_edges("github_comment", _route_after_github, {
    "slack_notify": "slack_notify",   # Critical 있으면
    "end": END,                        # 없으면 바로 끝
})
g.add_edge("slack_notify", END)
```

2-2. TypedDict State (노드끼리 주고받는 표준 상자)

비유: 부서 사이를 도는 **결재판**. 칸이 정해져 있고(이름/날짜/금액...), 각 부서가 자기 칸만 채운다.

개념: `AggregateState`는 파이프라인에서 노드끼리 주고받는 상태의 "칸 정의"다. Python의 `TypedDict`로 어떤 키가 들어갈지 미리 선언한다.

왜 쓰나 (이번에 실제로 겪은 함정): LangGraph는 **State 스키마에 정의된 키만** 노드 간에 전달한다. aggregate 노드가 `aggregated_issues`를 채워도, 만약 State에 그 키가 없으면 다음 노드로 안 넘어간다. 그래서 "aggregate가 쓰는 키"와 "slack_notify가 읽는 키"를 반드시 맞춰야 했다.

실제 코드 (`schema.py`):

```
class AggregateState(TypedDict, total=False):
    # 입력 (M0가 채움)
    task_id: str
    pr_id: str
    all_results: list[dict]
    # aggregate 노드가 채움
    aggregated_issues: list[dict]   # ← 이 키가 있어야 slack_notify가 읽음
    total_summary: dict
    # comment_refine 노드가 채움
    comment_body: str
    # github_comment 노드가 채움
    github_comment_url: Optional[str]
```

2-3. RAG 멀티에이전트의 "취합(aggregate)" 단계

비유: 여러 사람이 각자 조사한 보고서를 팀장이 **하나로 합치고 중요한 순서대로 정렬**해서 윗선에 올리는 일.

개념: 병렬로 실행된 여러 에이전트(M2 정적/보안 + M3 테스트/성능)의 출력(AgentOutput)을 모아 하나의 이슈 목록 + 요약으로 만드는 단계. **판정은 하지 않는다**(lgtm/rework는 Dev4 담당) — 데이터 정리·전달만 한다. 역할 경계를 지키는 게 중요.

왜 쓰나: 4개 에이전트 결과가 흩어진 채로 사람에게 가면 읽기 어렵다. Critical→Major→Minor 순으로 정렬해 한눈에 위험도를 보게 한다.

실제 코드 (`nodes/aggregate.py`):

```
_SEVERITY_ORDER = {"Critical": 0, "Major": 1, "Minor": 2}

all_issues, total = [], {"critical": 0, "major": 0, "minor": 0}
for result in state.get("all_results", []):
    agent = result.get("agent", "unknown")
    for issue in result.get("issues", []):
        all_issues.append({**issue, "agent": agent})   # 어느 에이전트가 찾았는지 태그
    for k in total:
        total[k] += result.get("summary", {}).get(k, 0)

sorted_issues = sorted(all_issues,
    key=lambda x: _SEVERITY_ORDER.get(x.get("severity", "Minor"), 3))
```

2-4. FastAPI BackgroundTasks (접수증 먼저, 일은 나중에)

비유: 택배 접수. 직원이 "접수됐습니다(번호 0042)"라고 **바로** 영수증을 주고, 실제 배송은 뒤에서 처리한다. 손님은 배송 끝날 때까지 창구에 묶여 있지 않는다.

개념: HTTP 요청에 즉시 `202 Accepted`로 응답하고, 무거운 작업(취합+LLM+GitHub+Slack)은 백그라운드에서 실행하는 패턴.

왜 쓰나: PR 분석은 수 초~수십 초 걸린다. 요청을 보낸 쪽(M0)이 그동안 기다리면 타임아웃·블로킹이 생긴다. "접수만 빠르게" 분리한다.

실제 코드 (`api/aggregate.py`):

```
@router.post("/aggregate", status_code=202)
async def trigger_aggregate(payload: AggregatePayload, background_tasks:
BackgroundTasks):
    background_tasks.add_task(run_aggregate, payload.model_dump())  # 뒤에서 실행
    return {"status": "accepted", "task_id": payload.task_id}       # 즉시 응답
```

2-5. Mock LLM 패턴 (개발 중엔 가짜로, 비용 0원)

비유: 요리 연습할 때 진짜 손님을 안 부르고 가족에게 시식시키는 것. 똑같은 동선을 공짜로 연습한다.

개념: `USE_MOCK_LLM=true`면 실제 Claude API를 호출하지 않고, 정해진 형식의 Mock 코멘트를 반환한다. 통합/데모 때만 `false`로 바꿔 실제 호출.

왜 쓰나: ① 개발 중 API 비용 절감 ② 테스트가 외부 API 없이 결정적으로 통과 ③ 실패 시 폴백(fallback)으로 파이프라인이 멈추지 않게.

실제 코드 (`nodes/comment_refine.py`):

```
def _use_mock_llm() -> bool:
    return os.getenv("USE_MOCK_LLM", "true").lower() == "true"
```

```
async def comment_refine_node(state):
    if _use_mock_llm() or ChatAnthropic is None:
        return {"comment_body": _build_mock_comment(state)}   # 가짜 코멘트
    try:
        llm = ChatAnthropic(model="claude-sonnet-4-6", temperature=0)
        response = await llm.ainvoke(prompt)
        return {"comment_body": response.content}
    except Exception:
        return {"comment_body": _build_mock_comment(state)}   # 실패해도 폴백
```

---

3. 구현 과정 (어떤 순서로, 어떻게 했나)

이번 작업은 "처음부터 만들기"가 아니라 **"구현계획을 보고 안 된 부분을 찾아 채우기"**였다. 과정:

1. **현황 파악**: 구현계획(3,794줄)과 실제 `dev2/` 폴더를 대조. 전체 테스트를 돌려 `127 passed`를 확인 → 파이프라인 1은 이미 완성, **파이프라인 2(Task 9·10·12)가 미구현**임을 특정.
2. **함정 발견**: `github_comment`/`slack_notify` 노드 파일은 있는데 **어떤 그래프에도 연결 안 된 "고아 파일"**이고 테스트도 없었다. → 재사용하기로 결정(중복 작성 방지).
3. **TDD 흐름으로 구현**: 스키마 → DB → 노드 → 그래프 → API 순서로 쌓고, 마지막에 테스트 7개 작성 → `7 passed`, 전체 `134 passed`.
4. **하위호환 수정**: 기존 `slack_notify`는 파이프라인 1의 `enriched` 키를 읽었다. 파이프라인 2는 `aggregated_issues`를 쓴다. → 둘 다 지원하도록 `state.get("aggregated_issues") or state.get("enriched", [])`로 안전하게 수정(기존 동작 안 깨짐).
5. **커밋·푸시**: 관련 파일만 골라 스테이징(무관한 pptx 임시파일 제외) → 커밋 → `task-interpretation-agent` 푸시 → 기존 PR #8(→ main)에 자동 포함.

핵심 설계 결정 (왜 이렇게 했나)

결정	이유
기존 github/slack 노드 **재사용**	계획은 PyGitHub로 새로 짜라 했으나 httpx 구현이 이미 있음 → 중복 방지
github → slack 순서	Slack 알림에 GitHub 코멘트 URL을 넣기 위해
DB 저장 실패는 **re-raise**, M4 push 실패는 **로그만**	핵심 기능(저장)은 멈춰야 하고, 비핵심(알림)은 전체를 막으면 안 됨 (CLAUDE.md §12)
SQL은 **파라미터 바인딩**만	f-string SQL 조합 금지 — SQL 인젝션 방지 (CLAUDE.md §6)

---

4. 프로젝트에서 — 파일별 역할 한눈에

파일	역할
`schema.py`	`AggregatePayload`(API 입력 계약), `AggregateState`(노드 간 상태) 정의
`migrations/004_aggregate_table.sql`	취합 결과 저장 테이블 `pr_aggregated_results` + 인덱스 3종

파일	역할
db.py	insert_aggregated_result() — 취합 결과 비동기 저장
nodes/aggregate.py	노드 I: 취합·정렬·저장·M4 push
prompts/comment_refine.py	수준별 코멘트 프롬프트 빌더(LEVEL_GUIDE)
nodes/comment_refine.py	노드 M: LLM 코멘트 생성 + Mock/폴백
nodes/github_comment.py	노드 N: GitHub PR 코멘트 등록(재사용)
nodes/slack_notify.py	노드 O: Critical Slack 알림(하위호환 수정)
aggregate_graph.py	그래프 조립 + 조건부 엣지 + run_aggregate 진입점
api/aggregate.py	POST /aggregate (202 비동기 접수)
tests/test_aggregate.py	단위 테스트 7개

---

5. 연관 개념 (더 공부하면 좋은 것)

- **conditional_edges** → 상태를 보고 다음 노드를 고르는 LangGraph의 핵심. Agentic Workflow의 본질.
- **멱등성(idempotency)** → 같은 PR을 두 번 취합하면? 지금은 매번 새 행 INSERT. 운영에선 UPSERT 고려 대상.
- **에러 처리 전략** → "삼키지 않기(핵심)" vs "로그만(비핵심)"의 구분. 분산 시스템 신뢰성의 기본.
- **계약(Contract) 기반 통신** → 에이전트끼리 Pydantic 모델로 입출력 형식을 고정 → 6명이 병렬 개발 가능.

---

6. 복습 퀴즈 (3문항)

1. **(이해)** comment_refine 다음에 github_comment를 실행하고, 그 뒤에 slack_notify를 두었다. 왜 Slack을 GitHub **뒤**에 두었을까?
2. **(적용)** 새 노드 kms_link를 만들어 comment_refine와 github_comment 사이에 끼우려 한다. 코드로 **무엇을 2가지** 바꿔야 하나? (힌트: 노드 등록 + 엣지)
3. **(비교)** aggregate 노드에서 DB 저장 실패는 raise로 위에 던지고, M4 push 실패는 로그만 남긴다. 이 차이의 기준은 무엇인가?

▶ 정답 보기

이해도 확인

이해가 되셨나요? 한 가지만 확인해볼게요:

> **Q.** "POST /aggregate가 202를 즉시 돌려주는데, 그럼 GitHub 코멘트가 실제로 달렸는지는 어떻게 확인할까요?" (힌트: 응답은 '접수증'일 뿐, 실제 작업은 백그라운드 → /insights 조회나 로그/DB로 확인)

더 깊이 테스트하고 싶으면 위 복습 퀴즈 3문항을 D+1, D+3, D+7에 다시 풀어보세요.